

Claude Prompt: Recreate the Reference Dhikr Counter App

Use the attached reference video as the primary visual and motion reference. Recreate the same overall product feel, composition, interaction model, and animation quality for my app. Do not make a generic prayer app or a static mockup; build a polished, responsive, functional mobile-first interface whose central experience is the animated counter.

Product concept

Create a focused dhikr/tasbeeh counter app for counting repetitions of a selected Arabic phrase. The interface should feel calm, premium, minimal, tactile, and meditative. The visual design is intentionally sparse: a nearly black canvas, a single luminous string of beads, restrained typography, and one clear action. The app should work well on modern phone screens and remain visually balanced on desktop widths.

Use these default content values, but keep them easy to replace through one configuration object:

- Arabic phrase: استغفر الله
- Transliteration: Astaghfar Allah
- Goal: 100
- Arabic goal label: ١٠٠ مرة
- English goal label: 100 Once
- Rounds label: الجولات
- Reset label: إعادة الضبط

Preserve proper right-to-left behavior for Arabic text. Arabic copy must render correctly, with appropriate line height, shaping, direction, and alignment. English transliteration and numerals may use a clean geometric sans-serif font such as Inter, SF Pro, or a close system equivalent. Use a modern readable Arabic font with a clean Naskh-inspired appearance, such as an available system or web font equivalent.

Visual direction

The page uses a full-screen deep-black background, approximately #000000, with no distracting gradients, illustrations, cards, or decorative panels. The visual hierarchy must be immediately clear:

1. The current count and round information are secondary but visible near the upper-right area.
2. The glowing active bead is the dominant focal point.
3. The dhikr phrase and supporting labels sit beneath the bead animation.
4. The reset control is the lowest-priority action at the bottom.

Use generous negative space. Do not overcrowd the screen. The design should feel like a quiet meditation tool rather than a dashboard.

Header / status area

Place a compact progress block in the upper-right safe area. Show the current progress as a large fraction, for example `23/100`, with the current count visually stronger than the goal. Beneath it, show the Arabic rounds label and value, for example `الجولات: 0`. Keep the block right-aligned and aligned to the RTL language context while ensuring the numbers remain easy to scan.

Use white text with subdued opacity for supporting information. Avoid heavy borders, filled cards, or excessive labels.

Bead centerpiece

Place a horizontal string of circular beads across the middle of the screen. The string follows a shallow downward-curving arc, like a section of a prayer bead strand viewed front-on. The beads should be evenly spaced, with a subtle connecting cord or very restrained implied connection if visible in the reference. Inactive beads are soft off-white or light gray. The active bead is brighter white and has a warm golden halo.

The active bead must be the visual anchor. Use a layered treatment: a bright circular bead, a soft radial golden glow behind it, and a subtle highlight that gives it a tactile, slightly dimensional feel without becoming glossy or 3D-heavy. Use approximately `#FFD700` or a slightly softened warm gold for the glow. Keep the glow controlled and elegant; it should not look like a neon game effect.

The bead strand must be clipped cleanly within the viewport. It should look continuous as beads move through the visible arc. Use a mathematically defined quadratic or cubic Bezier path, or an equivalent parametric curve, rather than manually positioning beads with unrelated coordinates.

Content block

Center the text beneath the beads. Display the Arabic phrase as the primary label in a bold, readable size. Place the transliteration below it in smaller, quieter text. Place the goal

information below that, using the Arabic and English labels as shown above. Maintain generous vertical rhythm and avoid excessive decoration.

Reset control

At the bottom center, add a transparent pill-shaped reset button with a thin white or white-with-opacity outline. Use the Arabic label `إعادة الضبط`. The button should be compact, calm, and secondary. On press, reset the count, rounds, bead position, and any transient feedback state with a short, polished transition. Add a subtle pressed-state opacity or scale change, but do not introduce a bright filled button.

Interaction and animation requirements

The main interaction is a tap/click on the bead area or the primary counting surface. Make the hit target generous and accessible, without changing the sparse visual composition. Each successful tap increments the current count by one and advances the bead strand.

Bead movement

When the user counts, animate the entire visible bead sequence along the curved path from right to left, matching the reference video. The active bead should travel into the active position while the next bead enters from the right and the previous bead exits on the left. Keep spacing stable during the movement so the strand feels physical and continuous.

Use an approximately `400ms` transition with a smooth ease-out curve, such as `cubic-bezier(0.25, 1, 0.5, 1)`. The movement should feel responsive and tactile: quick at the beginning, then gently settling into place. Avoid linear motion, excessive bounce, elastic overshoot, or abrupt teleportation.

The active bead should become brighter as it reaches the focal position. Animate the golden glow with a restrained pulse: briefly increase glow opacity and radius on arrival, then settle to a steady soft halo. The pulse must be synchronized with the bead arrival rather than looping continuously.

Count and feedback timing

Update the visible count when the bead finishes or nearly finishes its movement, so the number and bead position feel synchronized. Do not allow rapid repeated taps to corrupt the state. Either queue taps predictably or temporarily guard against duplicate input during the short transition.

On every successful count, show a small `+1` feedback label above the active bead. It should float upward approximately `20–30px` while fading from full opacity to zero over about

600ms . Use a subtle ease-out curve. The label should not permanently affect layout and should be removed after the animation completes.

If the platform supports it, trigger a light haptic feedback event on each successful count. Haptics are an enhancement, not a dependency; the app must remain fully functional without them.

Goal and rounds behavior

Track `currentCount` , `totalGoal` , and `rounds` in a clear state model. When `currentCount` reaches `totalGoal` , complete the round, increment `rounds` , and reset `currentCount` to zero or to the product decision that best matches the reference behavior. Make this behavior explicit and easy to change. At the goal boundary, use a restrained completion moment rather than confetti or a loud celebration: a slightly stronger glow, a brief calm transition, and an updated rounds value are sufficient.

Responsive behavior and accessibility

Design for a narrow mobile viewport first. Respect safe-area insets on iOS and Android. Keep the status block clear of notches and maintain comfortable tap targets. Scale bead spacing and the arc radius responsively so the strand remains centered without clipping on small devices. On wider screens, preserve the same visual intimacy rather than stretching the beads across the entire page.

Support keyboard activation and screen readers. Give the counting surface an accessible name such as `Count dhikr` . Give the reset button an accessible name in English and Arabic if appropriate. Maintain sufficient contrast for all essential text. Do not rely on color alone to communicate the active state.

Respect `prefers-reduced-motion` : replace the full bead slide and glow pulse with a short fade or a minimal position change while keeping the count interaction clear.

Implementation guidance

Use a clean component structure. Separate the counter state, bead-path calculation, bead rendering, feedback animation, progress header, dhikr content, and reset control. Keep all adjustable product values and visual constants centralized, including bead count, bead diameter, spacing, arc depth, active index, animation duration, glow colors, and goal value.

Use CSS transforms and opacity for animations so they remain performant. Prefer GPU-friendly transforms over layout-triggering properties. If using SVG, Canvas, or a DOM-based rendering approach, choose the simplest option that provides a smooth curved strand, responsive resizing, clean clipping, and precise animation control. Do not fake the arc with a static screenshot or a collection of unrelated absolute-positioned dots.

The final result should feel visually very close to the reference video while using original implementation code and configurable content. Match the reference's restraint, black-space composition, luminous active bead, right-aligned Arabic status area, centered phrase, outlined pill reset button, and tactile right-to-left bead motion.

Quality bar

Before considering the work complete, verify the following:

- The first screen immediately communicates a calm dhikr counter, not a generic dashboard.
- The active bead is unmistakably the focal point.
- The bead strand follows a smooth downward arc and stays continuous during movement.
- The count, rounds, and bead position remain synchronized.
- The `+1` feedback floats upward and fades without shifting the layout.
- The reset control is visually secondary and works correctly.
- Arabic text is rendered RTL and remains legible.
- The interface is responsive on small phone screens and wider screens.
- Reduced-motion behavior and keyboard/screen-reader basics are included.
- Avoid generic UI additions such as tabs, sidebars, cards, gradients, charts, excessive shadows, confetti, or unrelated navigation unless the product requirements explicitly demand them.

Start by implementing the core screen and interaction loop. Then refine spacing, typography, glow intensity, bead curvature, and easing against the reference video until the result feels calm, precise, and tactile.

Optional instruction for Claude

If you are generating code, first state the chosen stack and the component/state architecture in a short paragraph. Then implement the complete working screen rather than returning only a visual description. If any asset or font is unavailable, use a close system-safe fallback and preserve the layout and motion behavior. Do not ask me to manually fill in obvious details; make sensible assumptions and keep all key values easy to edit.